

Φ -Bench: Can Large Language Models Engineer the Infrastructure That Powers Them?

Leilei Ding^{*1‡}, Shumin Wang^{*1}, Yuting Huang^{*1}, Fanqi Wan², Yinmin Zhang², Qi Han², Yiming Xu³,
Feiyuan Zhang⁴, Xiaomeng Chu⁵, Guoliang You⁶, Wuyang Zhang¹, Daxin Jiang^{†2}, Yanyong Zhang^{†1}

¹ University of Science and Technology of China

² StepFun

³ Peking University

⁴ The Hong Kong University of Science and Technology

⁵ Yale University

⁶ University of Pennsylvania



Leaderboard



GitHub



Hugging Face

Abstract

Large language models (LLMs) have demonstrated remarkable capabilities in reasoning and code generation, raising the prospect that they could assist in developing and optimizing the very infrastructure that powers them. However, existing benchmarks mainly focus on isolated kernels, predefined operators, or pre-specified optimization targets, and therefore fail to evaluate the ability of LLMs to perform open-ended, long-horizon LLM infrastructure engineering. To address this gap, we present Φ -Bench, a benchmark for systematically evaluating LLMs on engineering the LLM infrastructure stack. Derived from optimization problems studied in frontier research and grounded in real-world code repositories, Φ -Bench provides broad coverage of the LLM infrastructure stack and spans tasks of varying complexity, ranging from localized kernel-level function completion to long-horizon implementation and end-to-end system optimization. Extensive experiments on frontier LLMs reveal their current capabilities and limitations in engineering complex LLM infrastructure, offering insights into the challenges that remain on the path toward autonomous optimization of future AI infrastructure.

Introduction

With recent advances in large language models (LLMs) in reasoning (Guo et al. 2025) and code generation (Z. AI 2026), leveraging LLMs to support the development of next-generation AI models has emerged as a promising research direction (Ma et al. 2026; Ishibashi, Yano, and Oyamada 2025). A particularly important challenge in this context is engineering and optimizing the software infrastructure underlying LLM training and inference, hereafter referred to as *LLM infrastructure*. Prior work (Kwon et al. 2023; Narayanan et al. 2021) has demonstrated that infrastructure-level optimizations can substantially improve GPU utilization and reduce computational costs. These developments naturally

raise an intriguing question: *Can large language models engineer and optimize the infrastructure that powers them?*

Although several benchmarks have evaluated LLMs on GPU kernel implementation (Ouyang et al. 2025; Li et al. 2025) and optimization (Nangia et al. 2026; Li et al. 2026), their evaluation settings are typically restricted to individual functions or small collections of isolated components. Consequently, they do not fully capture the complexity of engineering and optimizing real-world LLM infrastructure. In practice, such tasks are inherently long-horizon and open-ended: developers must understand and navigate an existing infrastructure stack, identify bottlenecks and optimization opportunities, and iteratively implement, profile, debug, and refine their solutions. This end-to-end workflow extends far beyond code completion or isolated, single-commit modifications. Furthermore, existing benchmarks cover only a narrow subset of the topics involved in LLM infrastructure engineering, limiting their ability to comprehensively assess whether an LLM can develop and optimize modern AI infrastructure. These limitations motivate the need for a comprehensive, long-horizon, and open-ended benchmark that more faithfully evaluates whether LLMs can engineer the infrastructure that powers them.

To bridge this gap, we introduce Φ -Bench, the Frontier AI Infrastructure Benchmark, as a systematic evaluation of frontier LLMs on real-world workloads derived from top-tier systems papers and public LLM infrastructure repositories. Three principles guide its design: long-horizon, open-ended problem solving, comprehensive coverage of LLM infrastructure, and scalable task synthesis. Unlike benchmarks centered only on completing isolated functions, Φ -Bench allows agents to navigate existing codebases and iteratively implement, profile, debug, and optimize their solutions.

To achieve broad coverage, we construct a bottom-up taxonomy of modern LLM infrastructure from 2,260 papers and 1,852 artifacts collected from public repositories, and use it to guide task synthesis. To scale benchmark construction beyond the limited supply of manually curated engineering artifacts, we develop an agent-loop-based pipeline that au-

[‡]Work done as an intern at StepFun.

^{*}Equal contribution.

[†]Correspondence to: Daxin Jiang <djiang@stepfun.com> and Yanyong Zhang <yanyongz@ustc.edu.cn>.

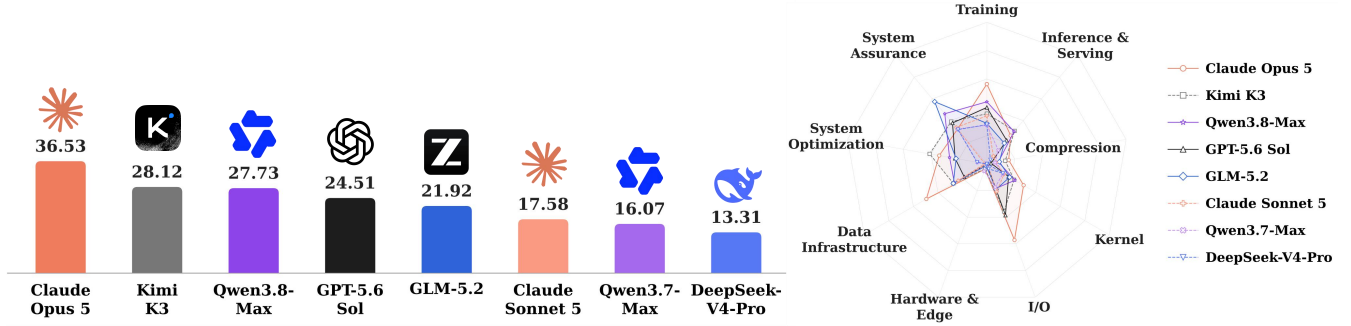


Figure 1: Performance of frontier models on Φ -Bench. **Left:** Overall scores achieved by different models. **Right:** Model performance across different infrastructure topics.

tomatically mines high-value challenges from repositories and iteratively generates comprehensive test cases. Together, these designs enable a broad, realistic, and scalable evaluation of LLM agents on infrastructure engineering.

The resulting benchmark comprises 85 challenging tasks in three task formats with progressively increasing scope and open-endedness: Kernel Function Completion (KFC), Long-Horizon Implementation (LHI), and End-to-End Optimization (E2EO). Together, these formats form a graduated evaluation spanning local kernel implementation, repository-scale infrastructure development, and end-to-end system optimization. This design enables Φ -Bench to distinguish an agent’s ability to implement efficient computational primitives, conduct long-horizon codebase engineering, and perform open-ended, hypothesis-driven infrastructure optimization.

Using Φ -Bench, we conduct a systematic evaluation of frontier LLMs. The best-performing model, Claude Opus 5, achieves a score of 36.53%, leaving substantial room for improvement. Further experiments show how the number of refinement iterations and the reasoning budget affect model performance. Detailed analyses of model solution trajectories further reveal their distinct strengths and weaknesses, providing insights into future model improvements, and highlight the key challenges that must be addressed before LLMs can reliably contribute to engineering and optimizing the infrastructure that powers them.

In summary, we make the following contributions:

- We introduce **Φ -Bench**, the Frontier AI Infrastructure Benchmark, comprising 85 challenging tasks grounded in real-world efforts to engineer and optimize the infrastructure for LLM training and inference.
- We develop a systematic, taxonomy-guided benchmark construction methodology grounded in research papers and repository artifacts and an agent-loop-based synthesis pipeline that automatically mines candidate engineering problems from repositories and constructs comprehensive test cases through iterative test generation.
- We systematically evaluate frontier LLMs on Φ -Bench and analyze their solution trajectories, revealing substantial room for improvement and providing insights into future model improvements.

Related Works

LLM Infrastructure Optimization

LLM infrastructure optimization aims to improve the performance, efficiency, and scalability of LLM training and inference across multiple layers. At the computation layer, CUTLASS (NVIDIA 2017) provides reusable building blocks for high-performance GPU kernels, while Triton (Tillet, Kung, and Cox 2019) offers a programming language and compiler for developing optimized GPU programs. Complementing these low-level abstractions, systems such as FlashAttention (Dao et al. 2022) and FlashInfer (Ye et al. 2025) further accelerate LLM training and inference by IO-aware attention algorithms, optimized inference kernels, and hardware-efficient execution.

Beyond individual kernels and runtimes, training frameworks such as Megatron-LM (Shoeybi et al. 2019) and DeepSpeed (Rasley et al. 2020) support parallel execution, memory partitioning, and communication optimization, whereas inference systems such as Orca (Yu et al. 2022), vLLM (Kwon et al. 2023), and SGLang (Zheng et al. 2024) provide KV-cache management, continuous batching, request scheduling, and distributed serving.

Benchmarks for LLM Infrastructure Engineering

Recent benchmarks have evaluated the ability of LLMs to implement and optimize components of LLM infrastructure (Lin et al. 2026; Wang et al. 2026). KernelBench (Ouyang et al. 2025), TritonBench (Li et al. 2025), and FlashInfer-Bench (Xing et al. 2026) primarily focus on individual GPU operators or fused operator compositions, with predefined interfaces, input-output specifications, and optimization objectives. Consequently, they do not require LLMs to navigate complete infrastructure repositories, identify performance bottlenecks, or coordinate modifications across multiple layers of the software stack. ISO-Bench (Nangia et al. 2026) and CUDAHercules (Li et al. 2026) extend this evaluation to repository-level GPU optimization, but their tasks typically specify the target component and performance bottleneck and therefore provide limited evidence of whether an LLM can resolve open-ended LLM infrastructure engineering challenges.

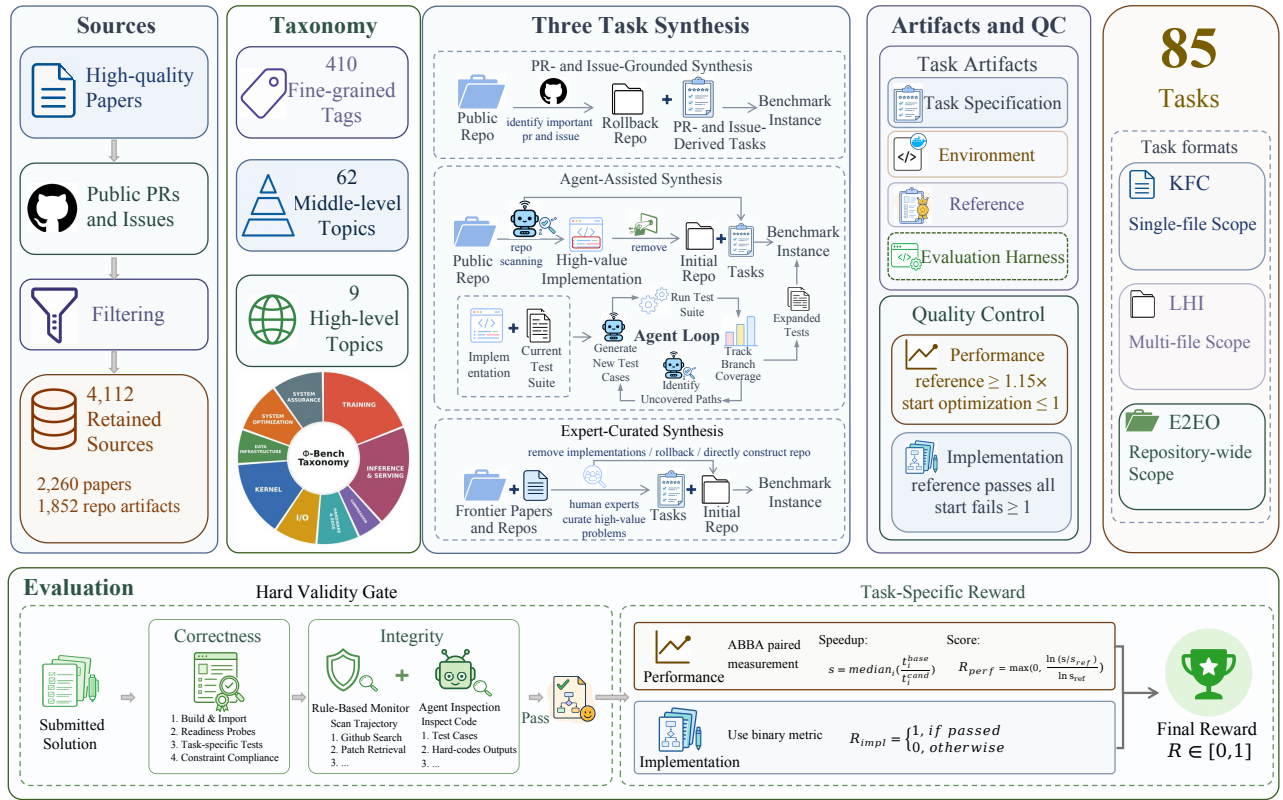


Figure 2: The task synthesis pipeline of Φ -Bench.

Benchmark Design and Construction

In this section, we describe the task formats, the construction process, and the evaluation metrics illustrated in Figure 2.

Task Formats

Each Φ -Bench task comprises a natural-language specification, a complete LLM infrastructure repository, executable workloads or test cases, and an evaluation harness. All materials, including the test cases, are visible to the agent except the evaluation harness. The specification defines either the required functionality or the performance objective.

Φ -Bench includes three task formats with increasing scope and open-endedness. KFC specifies the target function and its interface; LHI specifies a feature but leaves its implementation path open; and E2EO specifies only a system-level objective and constraints. Their editable and submission scopes expand accordingly, as summarized in Table 1.

Table 1: Comparison of the three task types.

Task Type	Implementation Objective	Editable Region	Submission Limit
KFC	Specified	Single file	Single
LHI	Specified	Multiple files	Multiple
E2EO	Free	Whole repository	Multiple

Kernel Function Completion (KFC). A KFC task isolates a performance-critical kernel or operator whose interface and input-output semantics are explicitly specified. The agent completes or optimizes its implementation within a single file without changing the external interface. Submissions are first tested for functional correctness and numerical accuracy, after which correct solutions are benchmarked for efficiency. KFC therefore evaluates the implementation of correct and efficient computational primitives.

Long-Horizon Implementation (LHI). An LHI task provides an issue-style feature request and a coarse-grained editable scope, such as a repository submodule, while leaving the relevant files, dependencies, and implementation strategy unspecified. Solving it requires navigating the codebase, understanding interactions among modules, modifying multiple files, and iteratively building, testing, and debugging the implementation. LHI thus evaluates nontrivial repository-level development rather than isolated function completion.

End-to-End Optimization (E2EO). An E2EO task provides a representative workload, a system-level optimization objective, and a set of constraints, without prescribing which components or strategies to use. The agent must profile the system, identify bottlenecks, formulate an optimization plan, and implement potentially repository-wide changes. E2EO therefore evaluates bottleneck discovery, cross-layer reasoning, and hypothesis-driven infrastructure optimization.

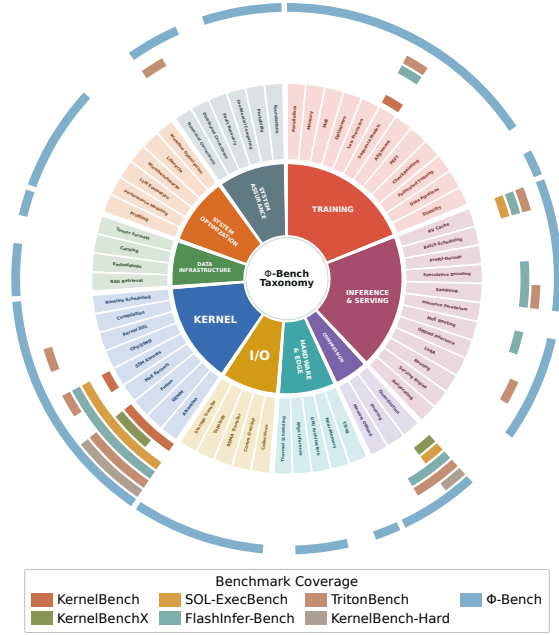


Figure 3: The high- and middle-level topics of our coverage taxonomy. The lines in the outer ring represent the topics covered by different benchmarks.

Task Synthesis

Φ -Bench tasks are derived from real-world research and engineering artifacts related to LLM infrastructure rather than from manually authored synthetic programming prompts. We collect papers published at top-tier systems conferences together with artifacts from public GitHub repositories implementing representative LLM infrastructure. We then construct a coverage taxonomy of LLM infrastructure engineering and synthesize tasks under its guidance.

Source collection and filtering. For the academic source pool, we collect papers published at top-tier systems conferences during the four-year period from 2023 to 2026. We then use an LLM-based filtering pipeline to retain papers that are directly related to LLM infrastructure, primarily study the implementation, acceleration, or resource optimization of LLM training or inference, and provide publicly available implementations. For the engineering source pool, we inspect issues and pull requests from public repositories of widely used LLM infrastructure. We similarly use LLMs to filter for nontrivial and technically challenging artifacts, prioritizing those that introduce substantial optimizations, add important infrastructure capabilities, address hardware- or workload-specific limitations, or document difficult engineering problems. This process yields 2,260 papers and 1,852 engineering artifacts.

Coverage taxonomy construction. We organize the selected sources into a hierarchical, three-level coverage taxonomy. For each source, we prompt a large language model to assign labels at the three predefined levels. The top level captures broad areas of LLM infrastructure engineering and

optimization. The middle level identifies more specific research problems, techniques, infrastructure components, and workload characteristics, while the bottom level consists of fine-grained tags describing the concrete problems addressed by the sources.

From the 4,112 sources described above, we extract 410 fine-grained tags capturing the concrete problems, techniques, and infrastructure characteristics represented in the source pool. We then cluster these tags into 62 middle-level topics, which are further organized into nine broad top-level categories. We use the middle-level topics as the primary index for task synthesis, while the provenance and characteristics of each source determine whether it is used to construct a KFC, LHI, or E2EO task. As illustrated in figure 3, the taxonomy helps Φ -Bench to achieve much more comprehensive coverage of LLM infrastructure engineering topics than any other related benchmarks.

Guided by the coverage taxonomy, we construct tasks through three complementary approaches: PR- and Issue-Grounded Synthesis, Agent-Assisted Synthesis, and Expert-Curated Synthesis. We describe each synthesis approach in detail below.

PR- and Issue-Grounded Synthesis. We identify important pull requests and issues from public LLM infrastructure repositories and reconstruct them as benchmark tasks. For each selected artifact, we use the repository state before the corresponding change as the task input and treat the implementation after the change as the reference solution. Whenever available, the associated unit tests are adapted to construct the evaluation harness. Small, well-scoped changes confined to a single file are typically converted into KFC tasks, whereas larger changes that span multiple files or infrastructure components are used to construct LHI or E2EO tasks. This procedure preserves both the real-world provenance of each task and the original engineering objective of the underlying repository change.

Agent-Assisted Synthesis. We employ agents to scan code repositories using a predefined synthesis pipeline and identify high-value implementation sites that can be converted into benchmark tasks. Depending on the scope and complexity of a selected site, we remove either a localized code segment or an entire implementation module, thereby creating tasks with different levels of implementation horizon and infrastructure context. To automate test generation, we further construct an agent loop that analyzes the execution flow of the target code. During execution, the agent tracks the branches exercised by the current test suite and iteratively generates additional test cases when it identifies previously uncovered execution paths. The resulting tests are used to evaluate whether a submitted implementation correctly handles the behaviors represented by the target code.

Expert-Curated Synthesis. For problems that cannot be reliably identified or reconstructed from repository changes alone, human experts curate challenging and high-value tasks from influential research papers and conference challenges. After identifying a target problem, the annotators construct an appropriate initial repository state by removing the rel-

High-Level Topic × Task Format									
KFC	15	7	2	19	5	2	1	3	1
LHI	1	2	4	8		1		1	3
E2E	3	2		1	2		1	1	
	TRAINING	INFERENCE & SERVING	COMPRESSION	KERNEL	I/O	HARDWARE & EDGE	INFRASTRUCTURE	DATA OPTIMIZATION	SYSTEM ASSURANCE

Figure 4: Task distribution in Φ -Bench.

evant implementation, reverting the repository to a simpler baseline, or, when no mature implementation is available, formulating the task directly from the problem specification. They then manually design test cases and evaluation workloads that precisely exercise the targeted capability. This approach is primarily used for problems that require substantial infrastructure-level reasoning, involve open-ended solution strategies, or represent emerging challenges for which repository history does not provide a complete reference trajectory.

Quality Control. We impose additional requirements during benchmark construction to ensure that the resulting evaluation is well posed and discriminative. For a task evaluated with the performance metric, the reference solution must achieve a stable performance of at least 1.15 in the official execution environment, whereas the starter or no-op solution must not outperform the baseline. For a task evaluated with the implementation metric, the reference solution must pass all test cases in the official environment, while the starter solution must fail at least one test.

Each task evaluated with the implementation metric contains at least five test cases covering multiple behavioral categories, including normal inputs, boundary conditions, error paths, and regression scenarios. A task whose reference solution fails these requirements is considered invalid and is revised or excluded rather than assigned a score.

Task Distribution

The final version of Φ -Bench contains 85 tasks, comprising 55 KFC tasks, 20 LHI tasks, and 10 E2EO tasks. Figure 4 summarizes their distribution across task formats and infrastructure topics. Φ -Bench covers all nine major infrastructure topic categories and, where permitted by repository characteristics, includes multiple task formats within each category.

Evaluation Metrics

Φ -Bench assigns each task attempt a reward in $[0, 1]$, with higher values indicating better performance. Across the three task formats defined above, we use two scoring metrics according to the evaluation objective of each task: a continuous performance metric for tasks with an explicit efficiency objective and a binary implementation metric for tasks evaluated primarily by functional completion. Both metrics are correctness-gated: buildability, functional correctness, edit constraints, and anti-cheating checks are evaluated before any reward is awarded.

Performance Metric. For tasks with an explicit performance objective, a candidate solution must first pass a set of hard validity checks, including static checks, editable region checks, and functionality checks. For a valid candidate, we measure its performance relative to the unmodified baseline using an AB-BA paired measurement protocol with at least five measurement pairs. Let t_i^{base} and t_i^{cand} denote the performance score of the baseline and candidate implementations in the i -th pair, respectively. The candidate performance is defined as

$$s = \text{median}_i \left(\frac{t_i^{\text{base}}}{t_i^{\text{cand}}} \right).$$

We obtain the reference performance s_{ref} by repeatedly evaluating the reference solution in the same execution environment and taking the median performance over at least five runs. A candidate receives a reward of zero if the reference solution does not yield a valid performance, i.e., $s_{\text{ref}} \leq 1$. Otherwise, the reward is computed as

$$R_{\text{perf}} = \begin{cases} 0, & s \leq s_{\text{ref}}, \\ \min \left(1, \frac{\ln(s/s_{\text{ref}})}{\ln s_{\text{ref}}} \right), & s > s_{\text{ref}}. \end{cases}$$

This logarithmic normalization assigns a reward of zero when the candidate matches or underperforms the reference solution. For candidates that outperform the reference, the reward increases logarithmically and reaches its maximum value of 1.0 when $s \geq s_{\text{ref}}^2$.

Implementation Metric. For tasks whose primary objective is functional implementation, we use a binary metric:

$$R_{\text{impl}} = \begin{cases} 1, & \text{if all conditions and test cases pass,} \\ 0, & \text{otherwise.} \end{cases}$$

A candidate receives full credit only if it builds and imports successfully, passes every test case, respects all task-specific forbidden-edit constraints.

Cheating Detection. To protect the integrity of Φ -Bench, we employ two complementary proctoring mechanisms: rule-based monitoring and agent-based inspection. The rule-based monitor scans the complete solution trajectory and final code changes for predefined prohibited behaviors, such as searching GitHub for the original implementation, retrieving the corresponding patch or commit diff and recovering upstream code from published Python packages. A dedicated proctor agent additionally inspects trajectories and submissions for detecting more complex hacking behaviors, such as hard-coding expected outputs, skipping or bypassing correctness checks, and introducing branches that artificially inflate the measured performance. If either proctoring mechanism detects a prohibited behavior, the corresponding task attempt receives a reward of zero.

Table 2: Category-wise performance on Φ -Bench. Scores are reported as percentages, with the best result in each category highlighted in bold. Models are ordered by their overall scores.

Model	Training	Inference & Serving	Compression	Kernel	I/O	Hardware & Edge	Data Infrastructure	System Optimization	System Assurance	Full
Claude Opus 5	56.40	26.20	15.70	30.00	57.10	3.90	49.40	34.20	32.20	36.53
Kimi K3	35.50	30.50	13.20	22.20	35.70	3.60	27.40	41.10	39.10	28.12
Qwen3.8 Max	43.90	29.90	8.30	23.00	18.30	0.10	26.50	26.80	46.30	27.73
GPT 5.6 Sol	39.90	21.80	3.00	17.70	38.50	0.00	18.40	22.80	37.80	24.51
GLM 5.2	28.70	19.20	9.00	18.60	19.10	1.40	27.40	22.10	57.40	21.92
Claude Sonnet 5	34.00	7.20	6.60	14.10	20.60	0.00	23.50	1.50	33.70	17.58
Qwen3.7 Max	27.40	12.70	5.90	13.20	17.40	5.40	0.00	6.80	32.20	16.07
DeepSeek V4Pro	27.80	3.90	0.50	13.10	3.70	0.90	0.00	7.00	31.70	13.31

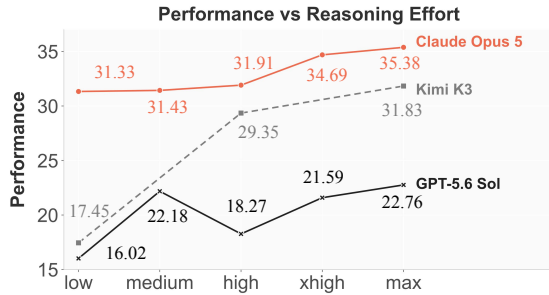


Figure 5: Performance over reasoning efforts on LHI tasks.

Evaluation

Experimental Setup

Models & Scaffolds. We evaluate a range of state-of-the-art proprietary and open-weight models on Φ -Bench. Proprietary models include claude-opus-5 (Anthropic 2026b), claude-sonnet-5 (Anthropic 2026c), gpt-5.6-sol (OpenAI 2026), qwen3.8-max (Qwen Team 2026b), and qwen3.7-max (Qwen Team 2026a). On the open-weight side, we evaluate kimi-k3 (Moonshot AI 2026), glm-5.2 (Z.AI 2026) and deepseek-v4-pro (DeepSeek-AI 2026). To evaluate each model under its strongest available inference configuration, we enable the highest reasoning setting exposed by the corresponding model and allow the maximum supported context length. We evaluate gpt-5.6-sol with its default scaffold, Codex (OpenAI 2025), and all other models with Claude Code (Anthropic 2026a) scaffold.

Implementation Details. We allocate 8 CPU cores, 32 GiB of memory, and a single NVIDIA H20 GPU to every task, while disabling all other GPU types to ensure a consistent evaluation environment. We allow each agent to submit a single submission for each KFC task and at most 16 candidate solutions for each LHI and E2EO task. We take the best submissions to compute the final score. Unless otherwise specified, the models were evaluated with reasoning effort uniformly set to max.

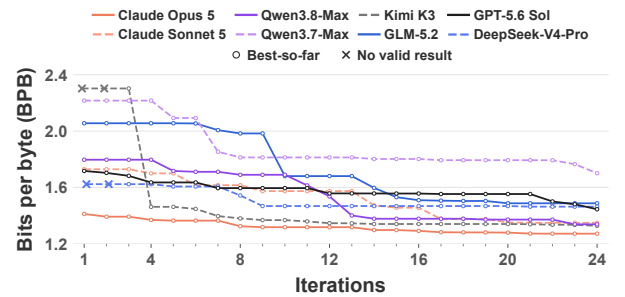


Figure 6: Performance curve of the best historical submission up to each round; cross marks indicate that the model has not yet produced a valid submission by the current round.

Main Results

Table 2 presents the performance of leading frontier models across the nine major categories of our benchmark. Claude Opus 5 achieves the highest overall score of 36.53%, followed by Kimi K3 at 28.12% and Qwen3.8 Max at 27.73%. Despite having access to the complete repository and test cases, even the strongest model achieves only slightly more than one-third of the maximum score, highlighting substantial headroom in real-world LLM infrastructure engineering.

Performance also varies considerably across infrastructure domains. Claude Opus 5 leads in five of the nine categories, demonstrating relatively broad competence, whereas other models exhibit more specialized strengths: Kimi K3 performs best on Inference & Serving and System Optimization, Qwen3.7 Max on Hardware & Edge, and GLM 5.2 on System Assurance. No model achieves uniformly strong performance across all categories. The Hardware & Edge category appears to be particularly challenging for the models: even the best-performing model achieved only 5.4%. This suggests that current frontier models still lack sufficient understanding of the hardware aspects of AI infrastructure and do not yet demonstrate strong engineering capabilities.

Ablation Studies

In this section, we investigate how model performance on LLM systems engineering tasks changes under different computational and interaction budgets. Specifically, we study two questions: (1) whether models can progressively improve their solutions through iterative implementation, evaluation, and refinement; and (2) whether increasing the test-time reasoning budget leads to better task performance.

Performance over iterations. To evaluate the models’ ability to iteratively refine their solutions, we select a challenging E2EO task from LLM training infrastructure. Given a fully editable nanoGPT training system, wall-clock budget, and parameter-count floor, the objective is to minimize validation bits per byte. Standard optimizations for this setting include efficient token dispatch (sorting, grouped batched GEMMs, per-expert capacity buffers), load balancing to prevent router collapse and token dropping, and routing configuration, and capacity factor — trading off quality, throughput, and drop rate.

The results in figure 6 show that most models exhibit certain ability to iteratively refine their solutions. Claude Opus 5 achieves a very low BPB score on its very first submission, and is able to continuously improve this metric over subsequent iterations. This result is highly impressive and clearly demonstrates its long-horizon reasoning and task execution capabilities. Although Qwen3.8-Max and Kimi K3 performed poorly in their early submissions, they are able to rapidly refine their approaches through iteration and ultimately achieved relatively low BPB scores. In contrast, DeepSeek V4Pro, Qwen3.7-Max, GLM5.2, and GPT5.6 Sol exhibit prolonged plateau periods during the iterative process and ultimately failed to optimize BPB to a low level.

Performance over reasoning budget. We evaluate three models including Claude Opus 5, Kimi K3, and GPT-5.6 Sol on 20 LHI problems across different reasoning effort levels, and plotted their scores as a function of effort level. The results in Figure 5 show that although all three models achieve their best performance at the max level, performance does not increase consistently with reasoning budget. In particular, GPT-5.6 sol exhibits notable instability at intermediate effort levels.

In addition, the performance gap between Claude Opus 5 and GPT-5.6 Sol across different budget settings is not particularly significant, whereas Kimi K3 loses about 45% of its score in the low setting compared with the max setting. This indicates that Kimi K3’s strong performance depends heavily on substantial test-time reasoning compute.

Analysis

Beyond the main evaluation, we collect detailed solution trajectories from different models and analyze them to identify their predominant error modes and detect potential cheating or hacking behaviors. This trajectory-level analysis provides more fine-grained insights into current model limitations and informs future improvements to their capabilities.

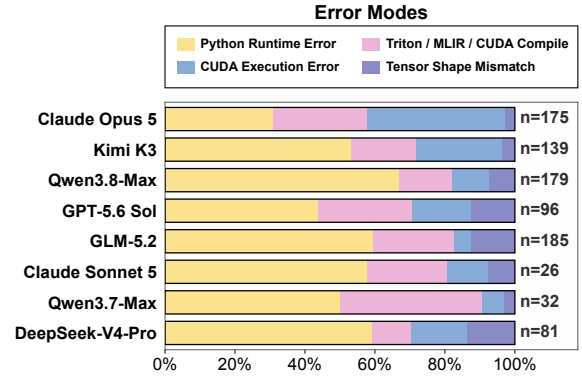


Figure 7: Error mode breakdown in model trajectories.

Error Modes Analysis

We collect error messages that appeared during each model’s problem-solving trajectories and categorize them into four major classes: Python runtime error, CUDA execution error, Triton / MLIR / CUDA compile error, and tensor shape mismatch. Figure 7 presents the number of errors and the distribution of error types across different models.

Most high-scoring models—Claude Opus 5, Kimi K3, and Qwen3.8 Max—produce more errors. This pattern suggests that they complete more challenging tasks through additional rounds of trial, diagnosis, and correction. In contrast, the weaker-performing models, including DeepSeek V4 Pro, Qwen3.7-Max, and Claude Sonnet 5, produced fewer total errors, suggesting that they tended to give up after a limited number of attempts or resort to simpler solutions. This contrast indicates that the ability to persist through trial and error and to iteratively correct mistakes is one of the core capabilities required for LLMs to handle challenging AI infrastructure engineering tasks.

In terms of error-type distribution, Python runtime errors account for more than half of all errors for most models. By contrast, Claude Opus 5 has a significantly lower proportion of this type of error than the other models, with a larger share of its errors concentrated in CUDA execution errors. This suggests that Claude Opus 5 is markedly better at generating Python code that works correctly within the repository on the first attempt, allowing it to focus more of its attention on the more critical low-level implementation and optimization issues. This is likely one of the reasons why it achieves a leading advantage in AI infrastructure engineering.

Capacity over open-endedness.

Table 3 compares model performance across three task categories with progressively increasing levels of open-endedness. Claude Opus 5 ranks first in every task format, achieving 37.16% on KFC, 21.60% on LHI, and 62.94% on E2EO. Kimi K3 ranks second overall and is particularly competitive on E2EO, while Qwen3.8 Max ranks third overall with consistently strong performance across all three task formats. Scores on LHI are consistently lower than those on KFC, indicating that long-horizon repository-level implementation remains challenging across all evaluated models.

Table 3: Performance across the three task formats in Φ -Bench. Scores are reported as percentages, with the best result in each column highlighted in bold. Models are ordered by their overall scores.

Model	KFC	LHI	E2EO	Full
Claude Opus 5	37.16	21.60	62.94	36.53
Kimi K3	26.09	19.55	56.41	28.12
Qwen3.8 Max	28.79	16.61	44.10	27.73
GPT 5.6 Sol	25.46	13.98	40.33	24.51
GLM 5.2	24.35	13.65	25.11	21.92
Claude Sonnet 5	18.14	13.46	22.74	17.58
Qwen3.7 Max	16.47	12.79	20.40	16.07
DeepSeek V4Pro	16.05	11.45	1.97	13.31

Hacking Prevention & Detection

In our experiments, we employed two approaches to minimize hacking behavior: (i) Soft network disconnection — we hijacked pip and URLs related to the test repositories, returning warning messages that block requesting such resources. (ii) Prompting — we prompt the models with hacking behavior definition and explicitly prohibit such actions.

Throughout all evaluation, the rule-based cheating detector identified only three instances in which DeepSeek V4 Pro triggered requests to retrieve code from the PyTorch website. The Proctor Agent confirmed that no actual hacking behavior occurred. This demonstrates that our two anti-hacking mechanisms effectively prevented cheating or hacking behavior during the evaluation process.

Case Study

Using the problem shown in Figure 6, we examine the optimization trajectories of three representative models: Claude Opus 5, the best-performing model on this task, and the substantially weaker Qwen3.7 Max and DeepSeek V4Pro. Figure 8 shows their per-round submission results. We ask a central question: *what behaviors enable strong models to effectively conduct iterative optimization of LLM infrastructure?* We view these trajectories through two coupled capabilities: selecting promising optimization directions and executing local changes reliably. The three observations below show how strong models connect these capabilities by establishing experimental foundations, controlling variables and noise, and cautiously attributing experimental outcomes.

Experimental Foundations

Observations. Claude Opus 5 establishes lightweight local validation experiments to rapidly screen candidate hypotheses before formal submission. These experiments reproduce only the relevant part of the workload and serve as inexpensive local checks. For example, Opus compares a short local run against the development measurement and observes only a small BPB difference, confirming that the local experiment is sufficiently informative for preliminary screening. It then uses such checks to reject unpromising changes before committing to full evaluation.

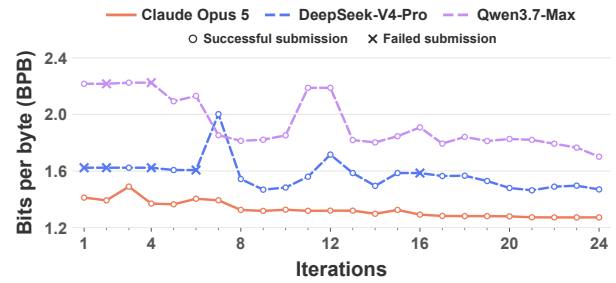


Figure 8: Performance of current submission at each round. Crosses denote submissions that failed the correctness check.

These local checks form part of a coherent search strategy: Opus explores candidate directions broadly before focusing on the most promising ones.

Qwen3.7 Max and DeepSeek V4Pro, by contrast, largely follow an implementation–submission–observation loop without preliminary local validation. Formal submissions therefore serve simultaneously as debugging, hypothesis testing, and evaluation, increasing the cost of unsuccessful hypotheses. For example, DeepSeek applies `torch.compile` to expert modules without first validating checkpoint compatibility, eventually wasting a submission because the resulting checkpoint cannot be loaded.

Takeaway. Good models establish low-cost mechanisms for hypothesis screening and validation, reducing the cost of each optimization iteration.

Variable and Noise Control

Observations. DeepSeek frequently modifies multiple variables simultaneously, making experimental outcomes difficult to attribute. Qwen3.7 Max exhibits the opposite failure mode: although it performs more disciplined single-variable tuning, many of its experiments produce differences comparable to measurement noise. Variable isolation alone is therefore insufficient when the observed effect is not reliably distinguishable.

Opus 5 explicitly considers both factors. It estimates measurement variation, accounts for differences in completed optimization steps, and focuses on experiments whose effects are sufficiently large to interpret. One representative transition changes the learning-rate schedule and worsens BPB from 1.3646 to 1.3932. Because the two runs use the same seed, complete the same number of steps, and have nearly identical runtime, Opus treats the degradation as reliable evidence rather than noise. This leads to the hypothesis that excessive time at very low learning rates is harmful, motivating the warmup–stable–decay schedule, which improves BPB to 1.3248. Rather than chasing each immediate improvement, Opus uses these results to update a consistent optimization plan.

Takeaway. Good models conduct meaningful experiments and maximize the information gained from controlling variables and noise in each iteration.

Cautious Attribution

Observations. Opus 5 also differs in how cautiously it interprets experimental outcomes. Rather than directly attributing a performance change to its latest modification, it actively checks for alternative explanations and confounding factors.

For example, one candidate modification initially appears to cause a substantial throughput regression. Opus identifies that the code change invalidated the Inductor compilation cache, causing the run to incur additional cold-compilation overhead. It therefore repeats the comparison under matched cache conditions instead of rejecting the modification. The corrected experiment shows essentially unchanged throughput while retaining a BPB improvement. Similar cache-related artifacts are detected multiple times in its trajectory, with Opus repeatedly delaying attribution until an appropriate control is available. This protects its overall optimization plan from being redirected by a local measurement artifact. In contrast, Qwen3.7 Max and DeepSeek V4Pro more often interpret observed changes directly in terms of the latest intervention.

Takeaway. Good models rigorously test alternative explanations before attributing experimental outcomes, preventing measurement artifacts from becoming incorrect optimization conclusions.

Overall, these trajectories suggest that long-horizon infrastructure optimization requires more than reliable local execution. Claude Opus 5 maintains a coherent optimization strategy across rounds: it explores broadly, uses controlled comparisons to identify promising directions, and updates its plan based on reliable evidence. The weaker models more often make local, greedy decisions, making it difficult for useful knowledge to accumulate across iterations.

Conclusion

We introduced Φ -Bench, a benchmark of 85 tasks spanning nine LLM infrastructure domains and three increasingly open-ended task formats. Its taxonomy-guided, agent-assisted construction pipeline grounds task synthesis in systems research and public repositories while enabling coverage and scalability. Evaluating eight frontier models reveals substantial but uneven capabilities: Claude Opus 5 leads with 36.53%, yet no model performs consistently well. On a challenging E2EO task, most models improve through iterative refinement; across 20 LHI tasks, greater reasoning effort does not yield consistent gains, although models perform best at the maximum effort level. Trajectory analysis further reveals notable differences in problem-solving capability and efficiency. Overall, Φ -Bench provides a realistic testbed for tracking progress toward more reliable and broadly capable LLM infrastructure agents.

References

- Anthropic. 2026a. Claude Code: Overview. <https://docs.anthropic.com/en/docs/claude-code/overview>. Official documentation.
- Anthropic. 2026b. Introducing Claude Opus 5. <https://www.anthropic.com/news/claude-opus-5>. Official model announcement.
- Anthropic. 2026c. Introducing Claude Sonnet 5. <https://www.anthropic.com/news/claude-sonnet-5>. Accessed: 2026-07-28.
- Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; and Ré, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.
- DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence.
- Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; and Zhang, e. a. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081).
- Ishibashi, Y.; Yano, T.; and Oyamada, M. 2025. Can Large Language Models Invent Algorithms to Improve Themselves?: Algorithm Discovery for Recursive Self-Improvement through Reinforcement Learning. [arXiv:2410.15639](https://arxiv.org/abs/2410.15639).
- Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J.; Zhang, H.; and Stoica, I. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, 611–626.
- Li, J.; Li, S.; Gao, Z.; Shi, Q.; Li, Y.; Wang, Z.; Huang, J.; WangHaojie, W.; Wang, J.; Han, X.; et al. 2025. Tritonbench: Benchmarking large language model capabilities for generating triton operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, 23053–23066.
- Li, S.; Zhang, Z.; Sun, G.; Luo, Y.; Chen, W.; Wang, Y.; Hong, M.; and Ding, C. 2026. CUDAHercules: Benchmarking Hardware-Aware Expert-level CUDA Optimization for LLMs. *arXiv preprint arXiv:2605.08467*.
- Lin, E.; Modi, S.; Hari, S. K. S.; Huang, Q.; Ye, Z.; Qin, N.; Zhou, F.; Zhang, Y.; Wang, J.; Damani, S.; et al. 2026. SOL-ExecBench: Speed-of-light benchmarking for real-world GPU kernels against hardware limits. *arXiv preprint arXiv:2603.19173*.
- Ma, Z.; Wang, G.; Xie, X.; Chen, Y.; Du, H.; Li, B.; Sun, Y.; Liu, W.; Chen, K.; and Li, Y. 2026. TREX: Automating LLM Fine-tuning via Agent-Driven Tree-based Exploration. [arXiv:2604.14116](https://arxiv.org/abs/2604.14116).
- Moonshot AI. 2026. Kimi K3: Open Frontier Intelligence. <https://www.kimi.com/blog/kimi-k3>. Official technical blog.
- Nangia, A.; Mishra, S.; Gokrani, A.; and Chopra, P. 2026. ISO-Bench: Can Coding Agents Optimize Real-World Inference Workloads? *arXiv preprint arXiv:2602.19594*.

- Narayanan, D.; Shoeybi, M.; Casper, J.; LeGresley, P.; Patwary, M.; Korthikanti, V. A.; Vainbrand, D.; Kashinkunti, P.; Bernauer, J.; Catanzaro, B.; Phanishayee, A.; and Zaharia, M. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21.
- NVIDIA. 2017. CUTLASS: CUDA Templates for Linear Algebra Subroutines. <https://github.com/NVIDIA/cutlass>.
- OpenAI. 2025. Introducing Codex. <https://openai.com/index/introducing-codex/>. Official product announcement.
- OpenAI. 2026. GPT-5.6: Frontier Intelligence That Scales with Your Ambition. <https://openai.com/index/gpt-5-6/>. Official model announcement.
- Ouyang, A.; Guo, S.; Arora, S.; Zhang, A. L.; Hu, W.; Ré, C.; and Mirhoseini, A. 2025. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*.
- Qwen Team. 2026a. Qwen3.7: The Agent Frontier. <https://qwen.ai/blog?id=qwen3.7>. Accessed: 2026-07-28.
- Qwen Team. 2026b. Qwen3.8-Max: A New Bar for Coding and Cowork. <https://qwen.ai/blog?id=qwen3.8>. Accessed: 2026-08-09.
- Rasley, J.; Rajbhandari, S.; Ruwase, O.; and He, Y. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 3505–3506.
- Shoeybi, M.; Patwary, M.; Puri, R.; LeGresley, P.; Casper, J.; and Catanzaro, B. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.
- Tillet, P.; Kung, H.-T.; and Cox, D. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 10–19.
- Wang, H.; Zhang, J.; Jiang, K.; Wang, H.; Chen, J.; and Zhu, J. 2026. Kernelbenchx: A comprehensive benchmark for evaluating llm-generated gpu kernels. *arXiv preprint arXiv:2605.04956*.
- Xing, S.; Zhai, Y.; Jiang, A.; Dong, Y.; Wu, Y.; Ye, Z.; Ruan, C.; Huang, Y.; Zhang, Y.; Yin, L.; et al. 2026. FlashInfer-Bench: Building the Virtuous Cycle for AI-driven LLM Systems. *arXiv preprint arXiv:2601.00227*.
- Ye, Z.; Chen, L.; Lai, R.; Lin, W.; Zhang, Y.; Wang, S.; Chen, T.; Kasikci, B.; Grover, V.; Krishnamurthy, A.; and Ceze, L. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of Machine Learning and Systems*.
- Yu, G.-I.; Jeong, J. S.; Kim, G.-W.; Kim, S.; and Chun, B.-G. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, 521–538.
- Z.AI. 2026. GLM-5: from Vibe Coding to Agentic Engineering. [arXiv:2602.15763](https://arxiv.org/abs/2602.15763).
- Zheng, L.; Yin, L.; Xie, Z.; Sun, C.; Huang, J.; Yu, C. H.; Cao, S.; Kozyrakis, C.; Stoica, I.; Gonzalez, J. E.; et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37: 62557–62583.